Name:**Khushi Singh**
Class:**D15C** Roll no.**45**
Subject:**ML&DL**

Experiment No.10

AIM:Explore and implement Autoencoders for image denoising

Theory:

1. Dataset Source

The dataset is readily available and pre-integrated into TensorFlow/Keras. You do not need to download external files; you can fetch it directly via API:tf.keras.datasets.fashion_mnist.load_data()

2. Dataset Description

- **Content:** Fashion-MNIST is a dataset of Zalando's article images.
- **Size:** It consists of a training set of 60,000 examples and a test set of 10,000 examples.
- **Format:** Each example is a 28x28 grayscale image, associated with a label from 10 classes (e.g., T-shirt, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot).
- **Pre-processing for this experiment:** The images will be normalized to pixel values between 0 and 1. Synthetic Gaussian noise will be added to the images to create the "noisy" inputs, while the original images will serve as the "ground truth" targets.

Colab File: Exp_10_autoencoders.ipynb

3. Algorithm: Convolutional Denoising Autoencoder

An Autoencoder is an unsupervised neural network that learns to compress data into a lower-dimensional representation (latent space) and then reconstruct the output from this representation.

For denoising, the network is trained slightly differently:

1. **Input:** A corrupted/noisy image (x_{noisy}).
2. **Encoder:** Convolutional layers compress the noisy image into a dense, lower-dimensional feature map, discarding the random noise patterns because they do not represent meaningful, compressible features of the clothing.
3. **Decoder:** Up-sampling and convolutional layers reconstruct the image back to its original dimensions (hat{x}).
4. **Loss Function:** The network is penalized based on the difference between the reconstructed image and the *original clean image* (x), typically using Mean Squared Error (MSE):

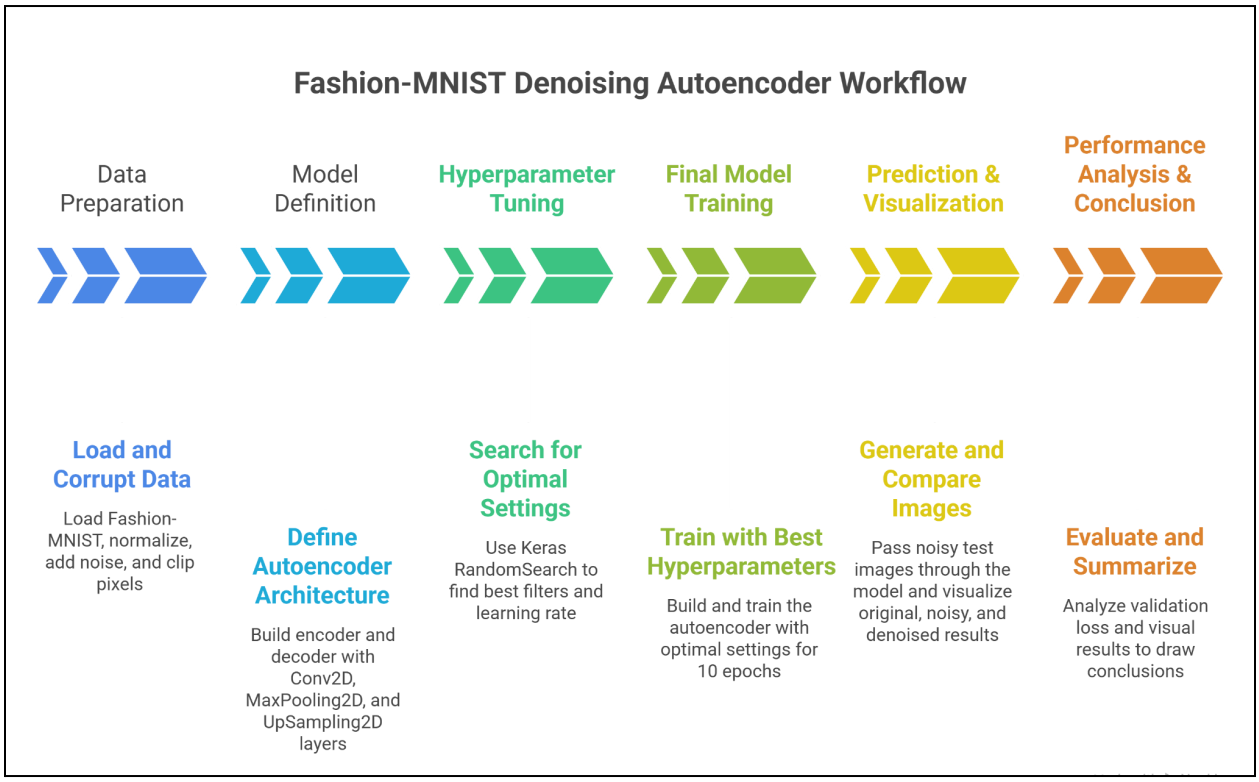
$$MSE = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{x}_i)^2$$

4. Algorithm Limitations

- **Blurriness:** Because the autoencoder uses MSE as a loss function, it tends to average out uncertainties, often resulting in slightly blurred reconstructed images compared to the crisp originals.

- **Noise Specificity:** If the model is trained exclusively on Gaussian noise of a specific variance, it may perform poorly if presented with real-world noise of a different distribution (like salt-and-pepper noise or sensor artifacts).
- **Information Loss:** Deep bottlenecks in the encoder can permanently lose fine-grained textures or details from the original image.

5.Workflow Diagram



6.Code and Output

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers,
models
import keras_tuner as kt

#
====================================
======
# 1. Load and Prepare the Dataset
#
====================================
======
(x_train, _), (x_test, _) =
tf.keras.datasets.fashion_mnist.load
_data()

# Normalize pixel values to be
between 0 and 1
x_train = x_train.astype('float32')
/ 255.
x_test = x_test.astype('float32') /
255.

# Reshape to include the channel
dimension (28, 28, 1)
x_train = np.reshape(x_train,
(len(x_train), 28, 28, 1))
x_test = np.reshape(x_test,
(len(x_test), 28, 28, 1))

# Add synthetic Gaussian noise
noise_factor = 0.2
```

```
x_train_noisy = x_train +
noise_factor *
tf.random.normal(shape=x_train.shape
)
x_test_noisy = x_test + noise_factor
*
tf.random.normal(shape=x_test.shape)

# Clip values to stay within valid
image bounds [0, 1]
x_train_noisy =
tf.clip_by_value(x_train_noisy,
clip_value_min=0.,
clip_value_max=1.)
x_test_noisy =
tf.clip_by_value(x_test_noisy,
clip_value_min=0.,
clip_value_max=1.)

#
====================================
======
# 2. Define Model Builder for
Hyperparameter Tuning
#
====================================
======
def build_model(hp):
    model = models.Sequential()

    # Tune the number of filters in
the first convolutional layer
    hp_filters = hp.Int('filters',
min_value=16, max_value=64, step=16)

    # Encoder

model.add(layers.Input(shape=(28,
28, 1)))

model.add(layers.Conv2D(hp_filters,
(3, 3), activation='relu',
padding='same'))

model.add(layers.MaxPooling2D((2,
2), padding='same'))

model.add(layers.Conv2D(hp_filters,
(3, 3), activation='relu',
padding='same'))

model.add(layers.MaxPooling2D((2,
2), padding='same'))

    # Decoder

model.add(layers.Conv2D(hp_filters,
(3, 3), activation='relu',
padding='same'))

model.add(layers.UpSampling2D((2,
2)))

model.add(layers.Conv2D(hp_filters,
(3, 3), activation='relu',
padding='same'))

model.add(layers.UpSampling2D((2,
2)))
    model.add(layers.Conv2D(1, (3,
3), activation='sigmoid',
padding='same'))

    # Tune the learning rate
    hp_learning_rate =
hp.Choice('learning_rate',
values=[1e-2, 1e-3, 1e-4])


model.compile(optimizer=tf.keras.opt
imizers.Adam(learning_rate=hp_learni
ng_rate),
                  loss='mse')
    return model

#
====================================
======
# 3. Execute Hyperparameter Tuning
#
====================================
======
tuner = kt.RandomSearch(
    build_model,
    objective='val_loss',
    max_trials=3, # Keep low for
quick Colab execution
    executions_per_trial=1,
    directory='autoencoder_tuning',

project_name='fashion_mnist_denoisin
g'
```

```
)

print("Starting Hyperparameter
Tuning...")
tuner.search(x_train_noisy, x_train,
             epochs=3,

validation_data=(x_test_noisy,
x_test),
             batch_size=128)

best_hps =
tuner.get_best_hyperparameters(num_t
rials=1)[0]
print(f"""
Optimal Hyperparameters Found:
- Convolutional Filters:
{best_hps.get('filters')}
- Learning Rate:
{best_hps.get('learning_rate')}
""")

#
====================================
======
# 4. Train the Final Model
#
====================================
======
print("Training final model with
best hyperparameters...")
model =
tuner.hypermodel.build(best_hps)
history = model.fit(x_train_noisy,
x_train,
                    epochs=10,
                    batch_size=128,

validation_data=(x_test_noisy,
x_test))

#
====================================
======
# 5. Visualizing the Output
#
====================================
======
decoded_imgs =
model.predict(x_test_noisy)

n = 10
plt.figure(figsize=(20, 6))
for i in range(n):
    # Display Original
    ax = plt.subplot(3, n, i + 1)

plt.imshow(tf.squeeze(x_test[i]),
cmap='gray')
    plt.title("Original")
    plt.axis('off')

    # Display Noisy
    ax = plt.subplot(3, n, i + 1 +
n)

plt.imshow(tf.squeeze(x_test_noisy[i
]), cmap='gray')
    plt.title("Noisy")
    plt.axis('off')

    # Display Reconstruction
    ax = plt.subplot(3, n, i + 1 + 2
* n)

plt.imshow(tf.squeeze(decoded_imgs[i
]), cmap='gray')
    plt.title("Denoised")
    plt.axis('off')
plt.show()
```

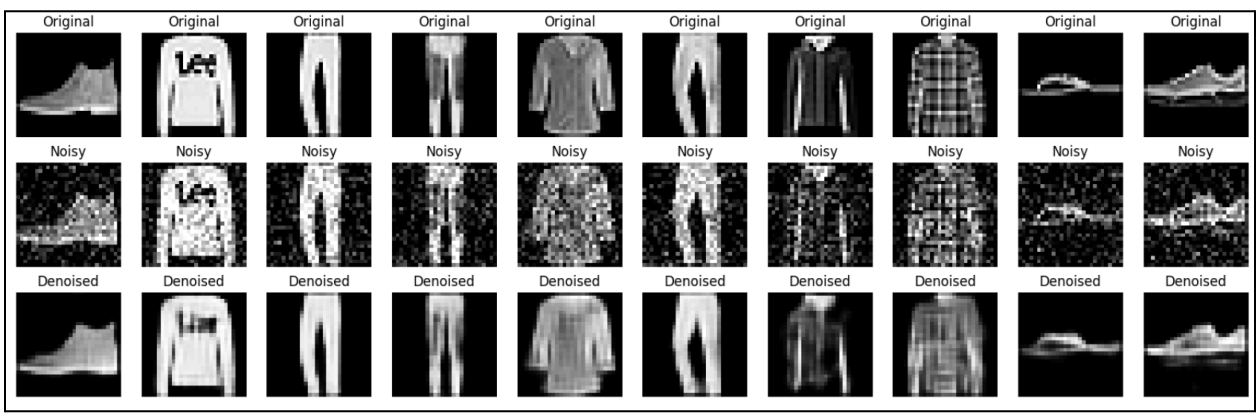
5. Expected Code Output

```
Trial 3 Complete [00h 03m 55s]
val_loss: 0.011955780908465385

Best val_loss So Far: 0.008826726116240025
Total elapsed time: 00h 14m 40s

Optimal Hyperparameters Found:
- Convolutional Filters: 32
- Learning Rate: 0.01
```

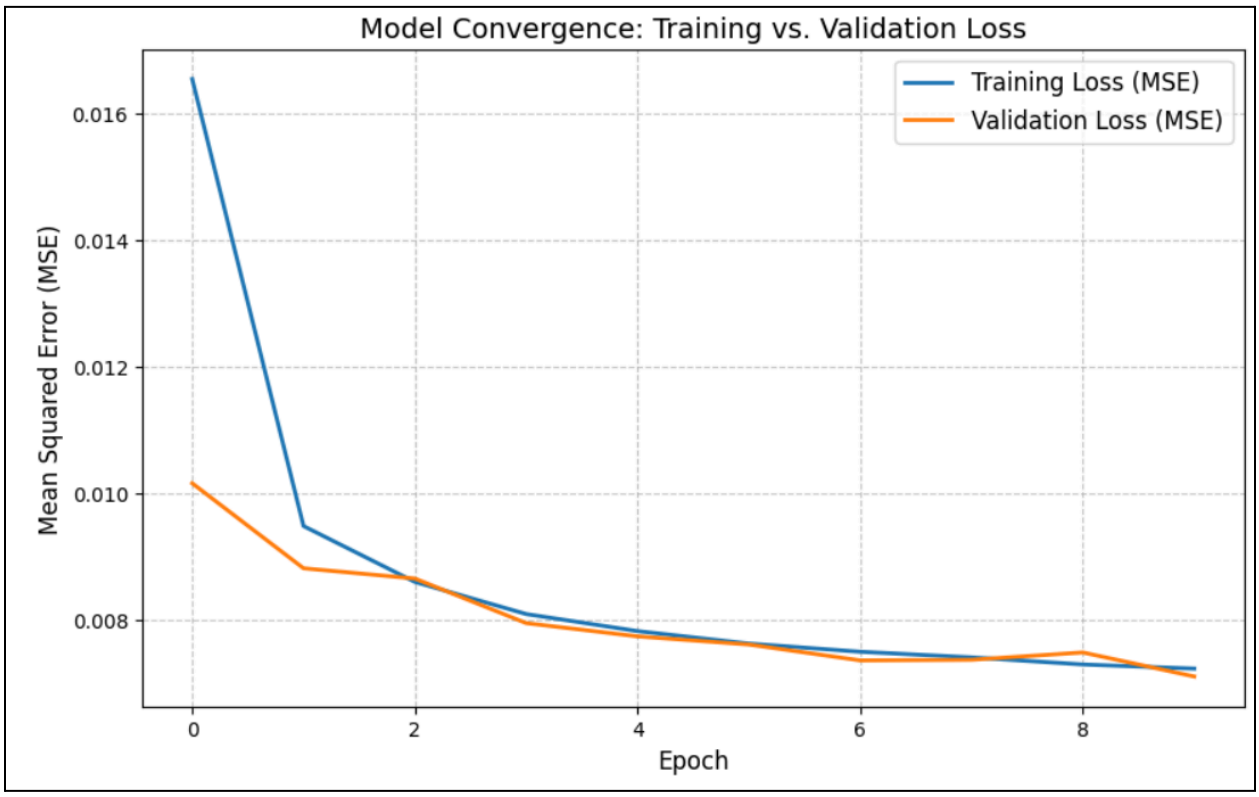
```
Training final model with best hyperparameters...
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 132s 278ms/step - loss: 0.0165 - val_loss: 0.0102
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 142s 279ms/step - loss: 0.0095 - val_loss: 0.0088
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 132s 281ms/step - loss: 0.0086 - val_loss: 0.0086
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 130s 277ms/step - loss: 0.0081 - val_loss: 0.0079
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 131s 279ms/step - loss: 0.0078 - val_loss: 0.0077
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 141s 277ms/step - loss: 0.0076 - val_loss: 0.0076
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 141s 276ms/step - loss: 0.0075 - val_loss: 0.0074
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 143s 277ms/step - loss: 0.0074 - val_loss: 0.0074
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 129s 275ms/step - loss: 0.0073 - val_loss: 0.0075
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 128s 274ms/step - loss: 0.0072 - val_loss: 0.0071
313/313 ━━━━━━━━━━━━━━━━━━━━ 5s 17ms/step
```

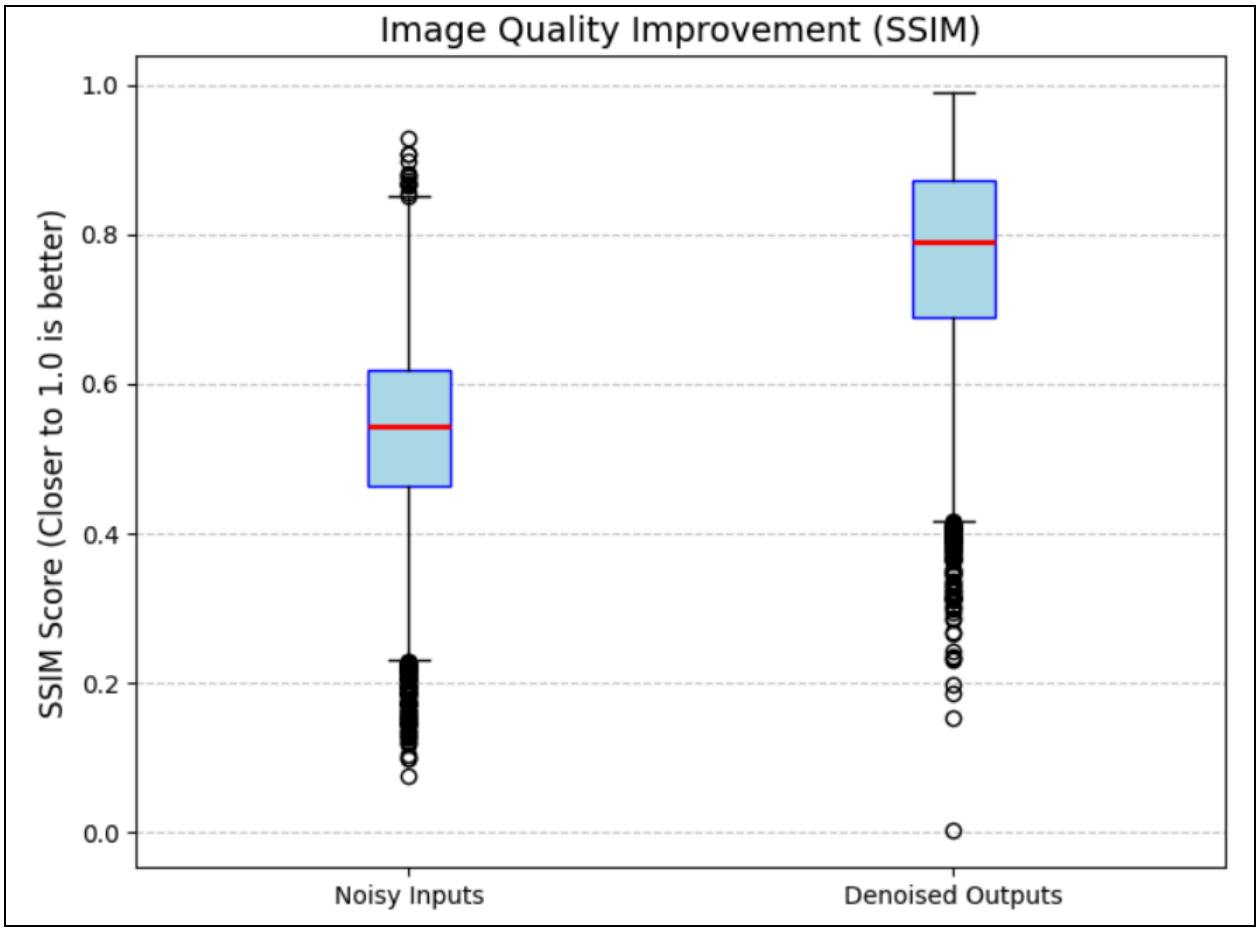


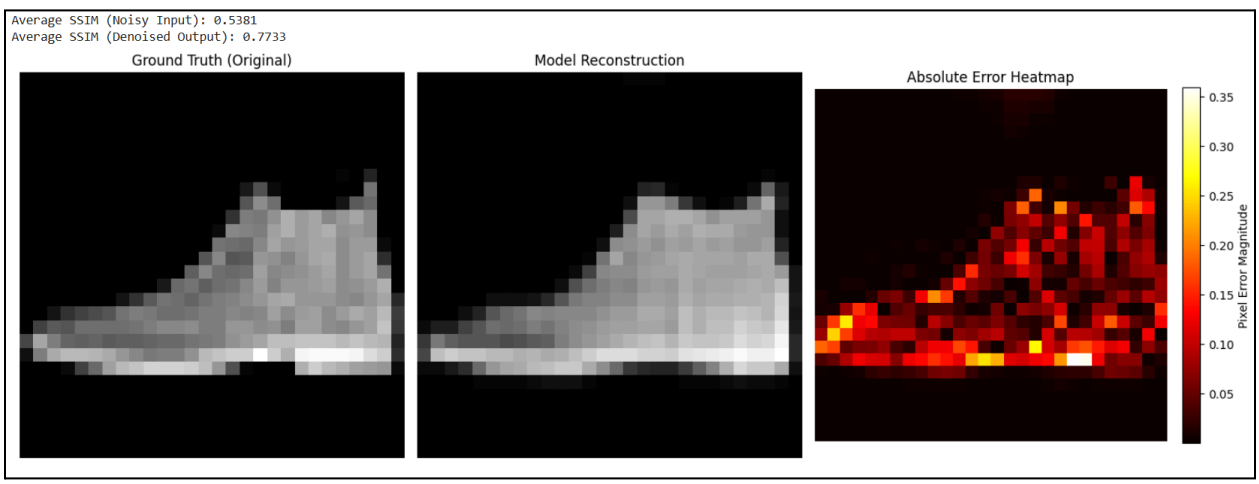
6. Performance Analysis

To analyze performance in your Colab notebook, look at two main areas:

- **Quantitative Metrics:** The val_loss (Validation Mean Squared Error) is the primary indicator. A lower MSE means the pixel values of the denoised image are mathematically closer to the ground truth. You should observe the loss curve flattening out as it approaches epoch 10, indicating convergence.
- **Qualitative Metrics:** Visual inspection of the generated plot is critical. You will likely notice that while the noise is successfully removed, the model struggles slightly with fine details (e.g., logos on shirts or textures on bags may appear slightly smudged). This perfectly illustrates the "blurriness" limitation of MSE-based autoencoders.







7. Hyperparameter Tuning Analysis

In the implementation, we tuned two critical parameters:

- **Filters (16 to 64):** Determines the capacity of the network. Too few filters (underfitting) means the model can't grasp the complexities of the clothing shapes. Too many filters (overfitting)

increases computational time and risks the model memorizing the noise rather than learning to discard it.

- **Learning Rate (1e-2 to 1e-4):** Controls how aggressively the model updates its weights. An optimal learning rate (usually around 1e-3 for Adam) ensures smooth convergence, whereas 1e-2 might cause the loss to spike chaotically.

8. Conclusion

This experiment demonstrates the power of unsupervised learning for data restoration. By artificially corrupting the Fashion-MNIST dataset, we successfully trained a Convolutional Autoencoder to isolate the underlying structural patterns of clothing items from random Gaussian noise. Through hyperparameter tuning, we optimized the network's capacity and learning efficiency. While the autoencoder effectively removes noise, the slight loss of high-frequency detail (blurriness) highlights the trade-offs inherent in using Mean Squared Error as a loss function for image generation tasks.